Problem 1

Is the exploration order what you would have expected? Is this a least-cost solution? If not, think about what depth-first search is doing wrong.

Yes, Depth-First Search (DFS) tends to commit to one direction and keep going until it hits a dead end or can't continue. Therefore, it produces a long and expensive exploration pattern as seen below in Figure 1. DFS does not minimize path cost, it only finds a path, not necessarily the shortest or cheapest path. The algorithm ignites path cost and ignores how close a state is to the goal. It uses Last-In First-Out (LIFO) rule, so the most recently discovered state is expanded next. This means it can go very far down a bad branch before considering better alternatives. In this maze, the algorithm is able to find a valid path, but it is not an optimal one.

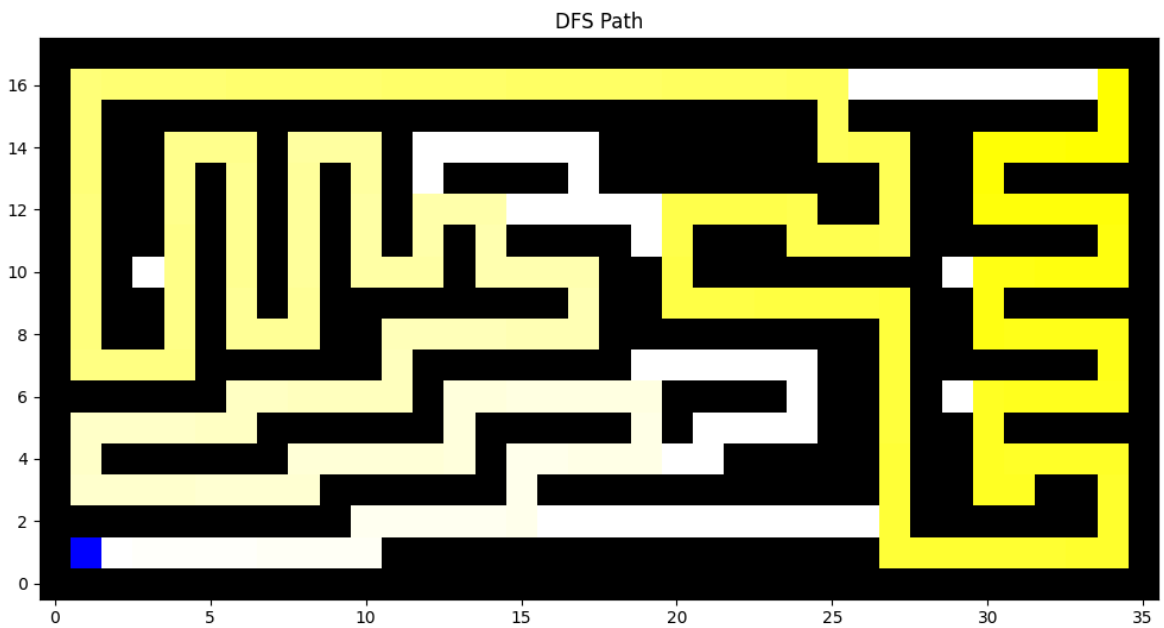
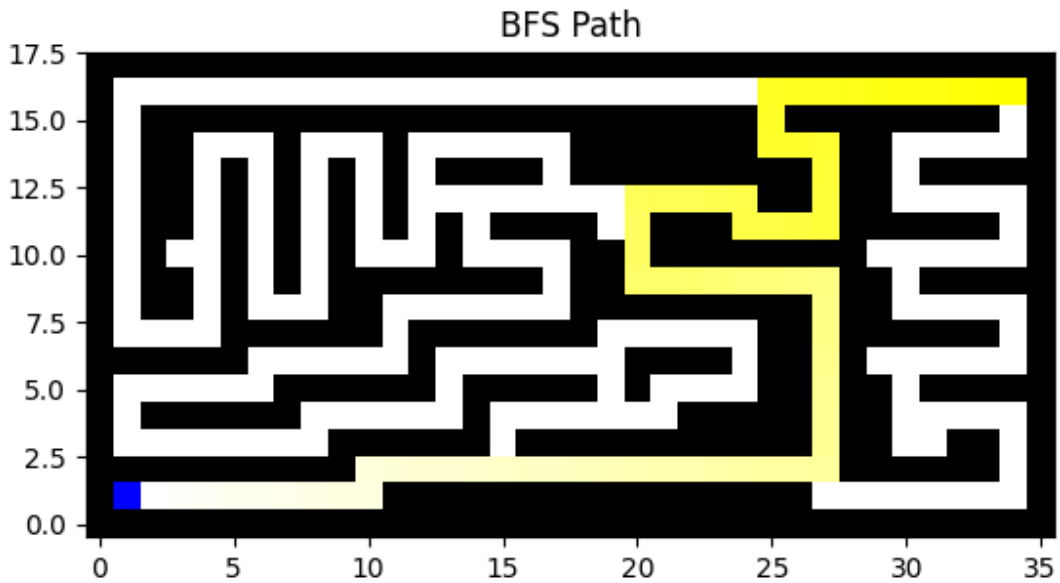


Figure 1: Depth-First Search (DFS) solution on mediumMaze. The long winding route illustrates that DFS explores deeply along one branch before backtracking, so it finds a valid path but not necessarily the least-cost or shortest solution.

DFS
cost = 224
visited nodes = 240
action length = 224

Problem 2



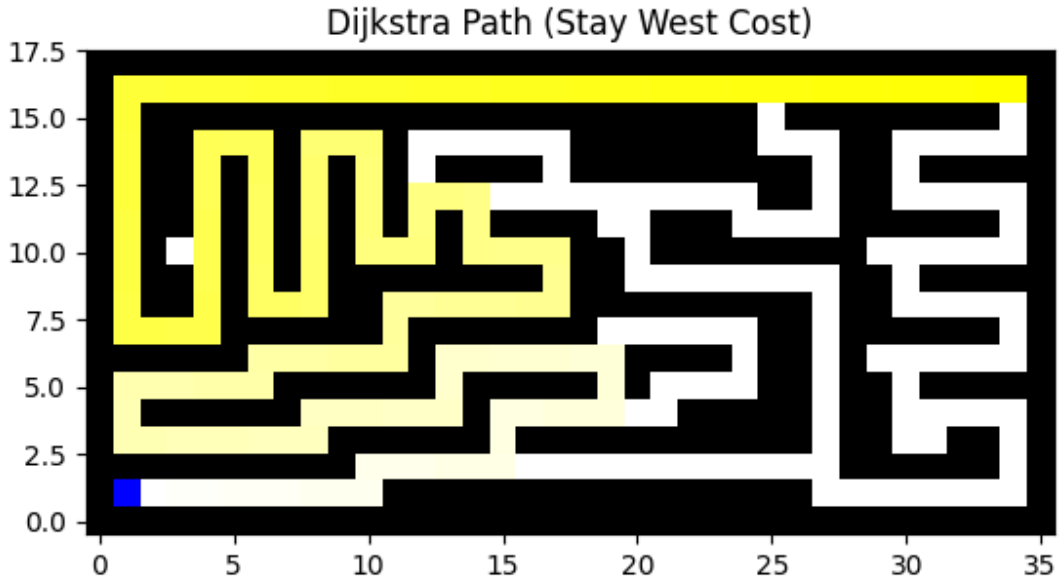
BFS

cost = 68

visited nodes = 223

action length = 68

Problem 3

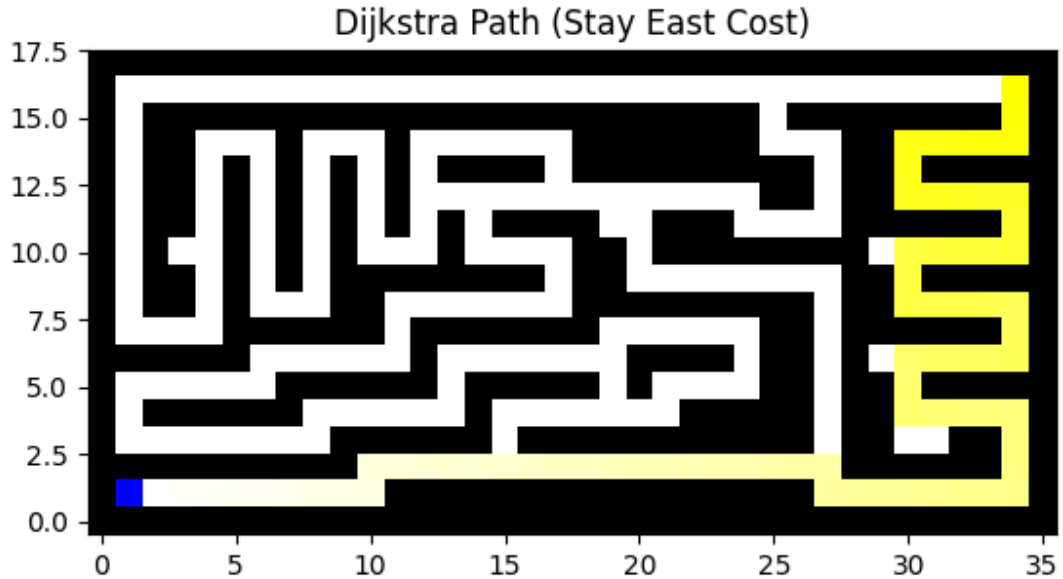


Dijkstra stayWestCost

cost = 25875

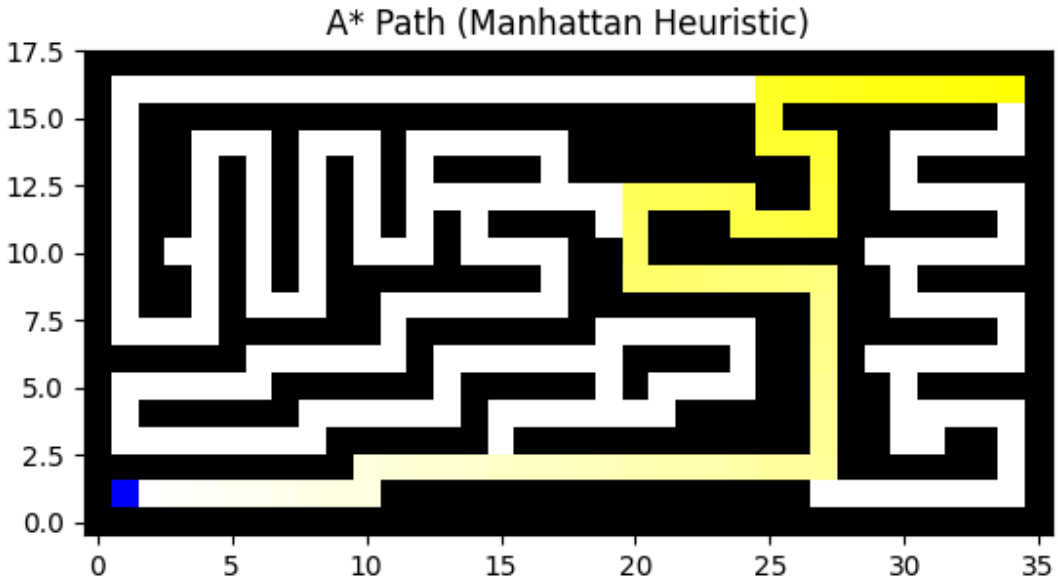
visited nodes = 246

action length = 152

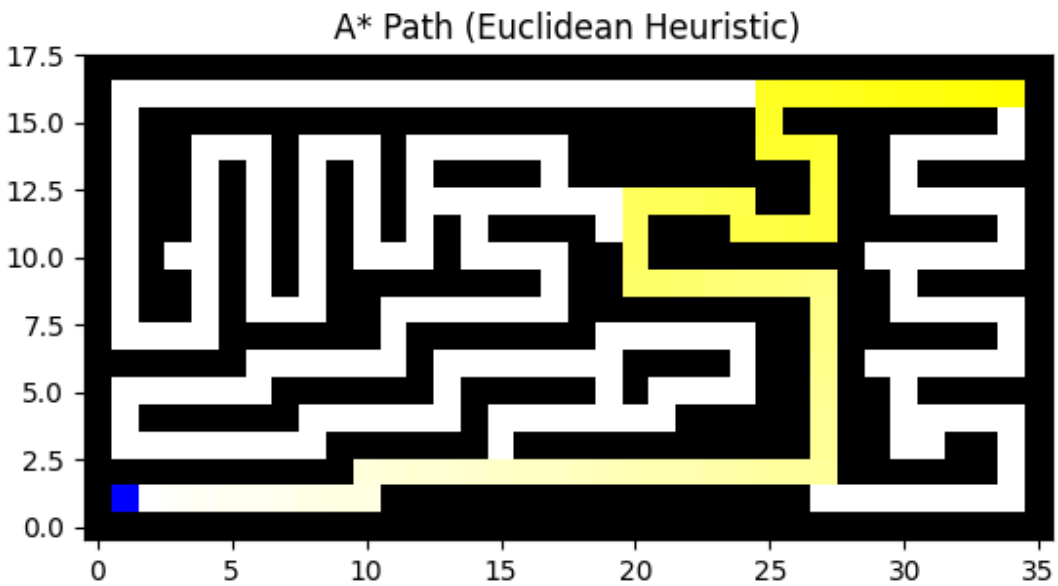


Dijkstra stayEastCost
cost = 13629
visited nodes = 101
action length = 74

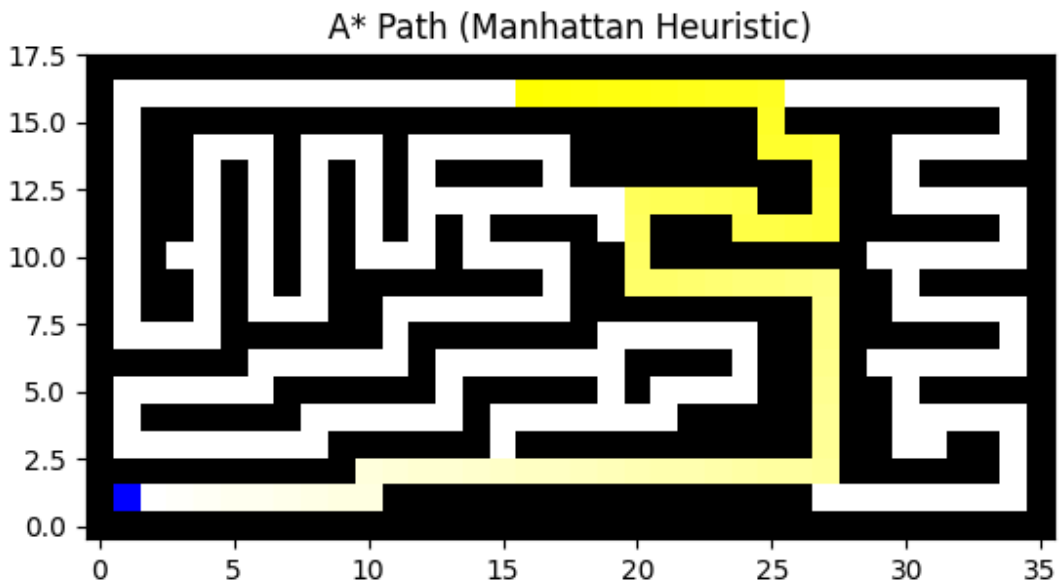
Problem 4



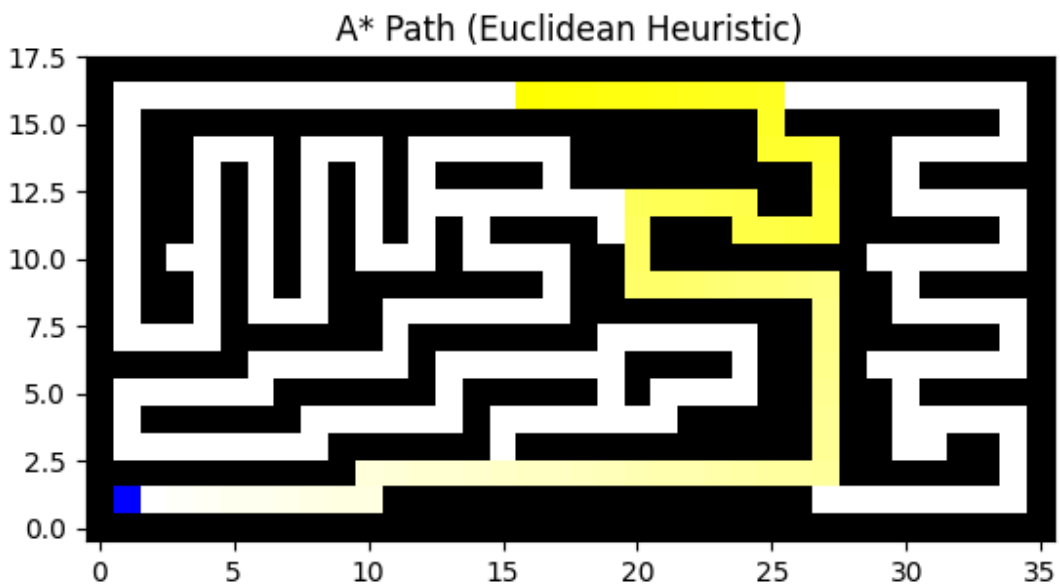
A* Manhattan
cost = 68
visited nodes = 146
action length = 68



A* Euclidean
cost = 68
visited nodes = 152
action length = 68



A* Manhattan
cost = 68
visited nodes = 161
action length = 68



A* Euclidean
cost = 68
visited nodes = 180
action length = 68

Compare the performance for the two different cases: Which heuristic is superior? Explain your answer.

Using two different goal states: $x_G = (34, 16)$ and $x_G = (16, 16)$. I found that A* with the Manhattan heuristic gave better performance than A* with the Euclidean heuristic. The Manhattan heuristic was superior because it is more consistent with the structure of the maze, where the robot can only move horizontally or vertically. The Euclidean heuristic still works, but because it assumes straight line distance it usually underestimates the cost more loosely and can lead to exploring more nodes than needed as seen in the above results of number of visited nodes. Therefore, Manhattan heuristic has a more efficient search.